



MODUL PEMROGRAMAN BERBASIS OBJEK 1

Pertemuan Ke-1

PHYTON BASE

Disusun Oleh :

Aziz Azindani, S.Pd., S,Kom., M.Kom

Pertemuan 1: Pengenalan Pemrograman Berbasis Objek (OOP) dan Kelas/Objek

Tujuan Pembelajaran:

- Memahami konsep dasar OOP
- Mengetahui cara membuat dan menggunakan kelas dan objek di Python

Materi Teori:

1.1. Apa itu Pemrograman Berbasis Objek (OOP)?

Pemrograman Berbasis Objek (Object-Oriented Programming, OOP) adalah paradigma pemrograman yang menggunakan "objek" sebagai unit dasar untuk membuat dan mengatur program. Objek-objek ini merepresentasikan entitas dunia nyata dan menyatukan data dan fungsionalitas dalam satu unit.

Contoh: Bayangkan sebuah objek `Mobil` yang memiliki atribut seperti `warna`, `merk`, dan `tahun`, serta metode seperti `jalan` dan `berhenti`. Semua ini dikelompokkan dalam satu entitas yang disebut objek.

OOP bertujuan untuk menyediakan cara untuk memodelkan hubungan kompleks dan struktur data yang lebih realistis, serta meningkatkan modularitas dan pengelolaan kode dengan memisahkan tanggung jawab.

1.2. Pemrograman Prosedural vs. OOP

- **Pemrograman Prosedural:**
 - Program dibangun sebagai serangkaian langkah atau prosedur.
 - Data dan fungsi dipisahkan; fungsi mengoperasikan data dari luar.
 - Contoh: Script yang mengambil data dari file dan menghitung rata-rata tanpa kaitan langsung antara data dan fungsi.
- **OOP:**
 - Program dibangun menggunakan objek yang menggabungkan data dan fungsi.
 - Data dan fungsi yang beroperasi pada data tersebut ditempatkan dalam unit yang sama (kelas).
 - Contoh: Sebuah kelas `Mahasiswa` yang menyimpan data tentang mahasiswa dan menyediakan metode untuk manipulasi data tersebut.

1.3. Empat Pilar OOP

1. **Enkapsulasi (Encapsulation):**
 - Proses menyembunyikan detail implementasi dari objek dan menyediakan antarmuka publik.
 - Melindungi data dengan memisahkannya melalui akses kontrol seperti `private` dan `protected`.
 - **Contoh:** Kelas `Mahasiswa` menyembunyikan atribut `npm` dari akses langsung dan menyediakan metode `get_npm()` untuk mengaksesnya.

2. Abstraksi (Abstraction):

- Proses menyederhanakan kompleksitas dengan menyembunyikan detail yang tidak penting dan hanya menampilkan fungsionalitas yang relevan.
- Menciptakan model atau antarmuka umum yang dapat diimplementasikan oleh objek yang berbeda.
- **Contoh:** Interface `Kendaraan` dengan metode `bergerak()` yang dapat diimplementasikan oleh berbagai jenis kendaraan.

3. Pewarisan (Inheritance):

- Proses di mana satu kelas dapat mewarisi atribut dan metode dari kelas lain.
- Memungkinkan pembentukan hierarki dan kode yang dapat digunakan kembali.
- **Contoh:** Kelas `Mobil` mewarisi atribut dan metode dari kelas `Kendaraan`.

4. Polimorfisme (Polymorphism):

- Kemampuan untuk memproses objek secara berbeda berdasarkan kelas atau tipe data mereka.
- Memungkinkan penggunaan antarmuka yang sama untuk tipe objek yang berbeda.
- **Contoh:** Metode `bergerak()` dapat diimplementasikan berbeda dalam kelas `Mobil` dan `Sepeda`.

2. Kelas dan Objek

2.1. Definisi Kelas dan Objek

• Kelas (Class):

- Kelas adalah blueprint atau template untuk membuat objek.
- Kelas mendefinisikan atribut (variabel) dan metode (fungsi) yang akan dimiliki oleh objek.
- **Sintaks:**

```
class NamaKelas:
    def __init__(self, parameter1, parameter2):
        self.atribut1 = parameter1
        self.atribut2 = parameter2

    def metode(self):
        # kode metode
```

• Objek (Object):

- Objek adalah instansiasi nyata dari kelas.
- Objek dibuat menggunakan kelas dan memiliki data serta fungsionalitas yang didefinisikan dalam kelas.
- **Sintaks:**

```
objek = NamaKelas(argumen1, argumen2)
```

Contoh:

```
class Mahasiswa:
    def __init__(self, nama, npm):
        self.nama = nama
        self.npm = npm

    def tampilkan_info(self):
        print(f>Nama: {self.nama}, NPM: {self.npm}")

# Membuat objek
mhs1 = Mahasiswa("Alice", "123456")
mhs1.tampilkan_info()
```

Dalam contoh di atas, `Mahasiswa` adalah kelas, sedangkan `mhs1` adalah objek dari kelas `Mahasiswa`.

2.2. Atribut dan Metode

- **Atribut (Attributes):**
 - Atribut adalah variabel yang dideklarasikan dalam kelas.
 - Atribut menyimpan data atau keadaan dari objek.
 - **Contoh:** `self.nama` dan `self.npm` adalah atribut dari kelas `Mahasiswa`.
- **Metode (Methods):**
 - Metode adalah fungsi yang dideklarasikan dalam kelas.
 - Metode mengoperasikan data atau keadaan dari objek.
 - **Contoh:** `tampilkan_info` adalah metode dari kelas `Mahasiswa`.

Penting: Dalam Python, semua atribut dan metode dalam kelas perlu diawali dengan parameter `self`, yang mereferensikan instansiasi objek saat ini.

2.3. Inisialisasi Objek dengan `__init__`

- Metode `__init__` adalah konstruktor di Python yang dipanggil secara otomatis saat objek dari kelas dibuat.
- Metode ini digunakan untuk menginisialisasi atribut objek dengan nilai awal.

Contoh:

```
class Mobil:
    def __init__(self, merk, model, tahun):
        self.merk = merk
        self.model = model
        self.tahun = tahun

# Membuat objek
mobil1 = Mobil("Toyota", "Corolla", 2020)
print(f">Mobil: {mobil1.merk} {mobil1.model}, Tahun: {mobil1.tahun}")
```

2.4. Contoh Sederhana: Kelas Mahasiswa

```
python
Copy code
class Mahasiswa:
    def __init__(self, nama, npm):
        self.nama = nama
        self.npm = npm

    def tampilkan_info(self):
        print(f>Nama: {self.nama}, NPM: {self.npm}")

# Membuat objek
mhs1 = Mahasiswa("Alice", "123456")
mhs1.tampilkan_info()
```

Penjelasan:

- Kelas Mahasiswa memiliki dua atribut: nama dan npm.
- Metode `__init__` menginisialisasi objek dengan nama dan NPM.
- Metode `tampilkan_info` menampilkan informasi mahasiswa.

2.5. Praktik Mandiri

Tugas:

1. Buat kelas `Mobil` dengan atribut `merk`, `model`, dan `tahun`.
2. Tambahkan metode untuk menampilkan informasi mobil.

Contoh Solusi:

```
python
Copy code
class Mobil:
    def __init__(self, merk, model, tahun):
        self.merk = merk
        self.model = model
        self.tahun = tahun

    def tampilkan_info(self):
        print(f>Merk: {self.merk}, Model: {self.model}, Tahun: {self.tahun}")

# Membuat objek
mobil1 = Mobil("Toyota", "Corolla", 2020)
mobil1.tampilkan_info()
```

2.6. Latihan

Tugas:

- Buat kelas `Buku` dengan atribut `judul`, `penulis`, dan `tahun_terbit`.
- Tambahkan metode untuk menampilkan informasi buku.